

A Query Recommender for Preference-Aware Natural Language Querying

Implementation Report — Phase 4 (Query Recommender Core)

The thesis's central contribution: retrieval, LLM generation, schema-grounding, and behavior-aware ranking

Narges

Supervisor: Prof. Elisa Quintarelli

July 28, 2026

Abstract

This report documents Phase 4 of the thesis — **the thesis's actual contribution**: turning the empty `suggestions: []` stub left by Phase 2 into a working query recommender. Four stages are implemented and composed inside the same shared entry point Phase 2 already exposed, `handle_recommend()`, so no new integration surface was needed: (4a) retrieval of a user's own semantically similar past queries; (4b) LLM-based candidate generation, conditioned on both that retrieved history *and* Phase 3's stated-vs-actual behavioral divergence signal; (4c) a schema-grounding validation step, whose originally-planned precision design had to be abandoned after an empirical calibration showed it would reject the supervisor's own canonical example query; and (4d) a two-axis ranking-and-diversification step that is the concrete mechanism through which the behavioral signal shapes the final three suggestions. The full chain was verified end-to-end, first with a mocked LLM, then over real HTTP with a placeholder API key, and finally — after both configured cloud LLM keys turned out to be unusable for account/billing reasons — against genuine model output from a locally hosted Ollama model, producing suggestions that concretely demonstrate the thesis's central premise.

Contents

1	Introduction	3
1.1	Scope	3
1.2	Where this fits	3
2	High-Level Overview	4
2.1	The core idea	4
2.2	Pipeline diagram	4
3	4a — Query-Log Retrieval	6
4	4b — Candidate Generation	6
4.1	Prompt construction	6
4.2	LLM abstraction reuse	7
5	4c — Schema-Grounding Validation	8
5.1	What the roadmap originally specified	8
5.2	Why the first two implementations were rejected	8
5.3	What shipped instead	9

6	4d — Ranking and Diversity	10
6.1	Scoring	10
6.2	Diversification	10
7	End-to-End Verification	10
7.1	Full chain, mocked LLM	10
7.2	Full chain, real HTTP, placeholder key	10
8	Real-LLM Verification: Gemini/OpenAI Blocked, Ollama Used Instead	12
8.1	Verified against real output	12
9	Interpretation and Limitations	12
10	Summary and Next Steps	13

1 Introduction

Phases 0–3 built, in order: two running Virtual Knowledge Graphs and a domain-switching harness (Phase 0/1, an internal development tool, not a thesis deliverable), a fixed API contract and a persistent per-user query log (Phase 2), and a per-user behavioral divergence signal quantifying how a user’s *stated* search queries diverge from what they *actually* watch (Phase 3). None of these, individually, recommend anything — Phase 2’s endpoint always returned an empty suggestion list. Phase 4 is where the professor’s actual request is implemented: given a query a user just asked, and given both their query history and their behavioral profile, propose three follow-up queries an autocomplete-style, history-only recommender would not think to suggest.

1.1 Scope

Phase 4 implements the four-stage pipeline the project roadmap specifies:

Stage	Role
4a	Retrieval — this user’s past queries most similar to the current one
4b	Generation — an LLM proposes a pool of candidate follow-up queries
4c	Validation — drop candidates with no plausible link to the domain schema
4d	Ranking — score and diversify what remains down to 3 final suggestions

All four stages live in a new `recommender/core/` package. No new API surface or harness wiring was required: Phase 2’s `handle_recommend()` already sat at the point where a request (query text, user id, domain) becomes a response (a list of suggestions); Phase 4 simply replaces the hardcoded empty list with a real call into this pipeline.

1.2 Where this fits

This is the thesis’s core contribution, in the sense used throughout the project roadmap (§2a): everything before this phase built the harness, the API shell, and the behavioral-signal input this phase consumes; everything after it (Phase 5) evaluates what this phase produces.

2 High-Level Overview

2.1 The core idea

The central research question, attributed by the supervisor’s email to “Jerry’s” analysis, is that a user’s stated queries and their real interaction behavior often diverge, and that a recommender conditioning on query history alone is little more than autocomplete. Phase 4 operationalizes this directly: stage 4b’s prompt explicitly presents the LLM with two separate signals — what the user typed before, and what their behavioral profile says they actually engage with — framed as potentially *different* things, not two views of the same preference. Stage 4d then makes this signal load-bearing rather than decorative: one of its two scoring axes rewards candidates that textually align with the behavioral profile’s real top categories, not merely with the user’s own past phrasing.

2.2 Pipeline diagram

Figure 1 shows the full chain, from the two upstream inputs (Phase 2’s query log and Phase 3’s behavioral profile) through to the three suggestions returned by `handle_recommend()`.

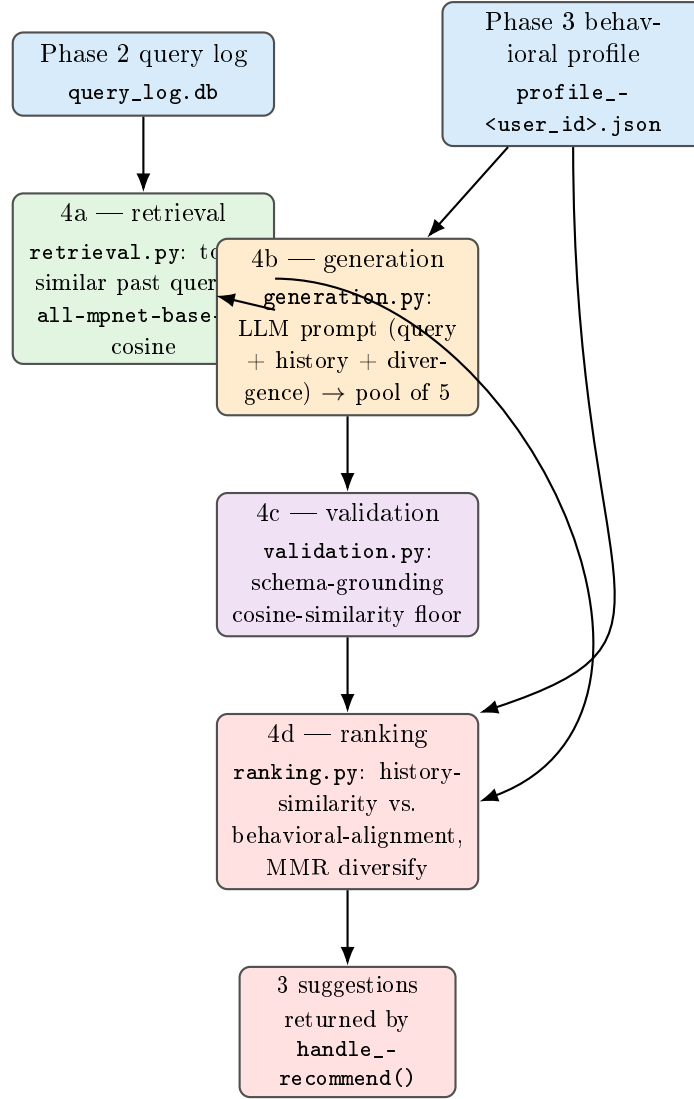


Figure 1: Phase 4 data flow. The behavioral profile (right) feeds both 4b’s generation prompt and 4d’s ranking score directly — the two concrete points where the thesis’s stated-vs-actual signal shapes the final output, not just the initial candidate pool.

3 4a — Query-Log Retrieval

`retrieve_similar_queries(user_id, query_text, domain=None, top_k=3, history_limit=200)` implements the retrieval half of RA-GQR (Bacciu et al., IR-RAG@SIGIR 2024):

1. Pulls the user’s query history from the Phase 2 SQLite log (`recommender.query_log.get_user_history()`), optionally scoped to one domain.
2. Excludes exact, case-insensitive duplicates of the current query.
3. Embeds the current query and every remaining history entry with `all-mpnet-base-v2 (sentence-transformers)` — the same model already used elsewhere in `web_app` for embedding-based class/property extraction, reused rather than introducing a second model dependency.
4. Ranks candidates by cosine similarity (embeddings are pre-normalized, so this reduces to a plain dot product) and returns the top-*k*, each annotated with its similarity score.

The retrieved queries become the few-shot/context material for 4b’s prompt, in place of generically hand-curated examples — directly implementing RA-GQR’s central idea that retrieval from a user’s own log outperforms static examples.

Cold start. If a user has no other logged queries, the function returns an empty list; 4b’s prompt formatter renders this as an explicit “no prior queries from this user” note and still functions correctly, consistent with GQR’s original premise that the technique does not require historical data to work at all.

Verification. `user_00001`’s query log was seeded with 4 varied movie queries. Calling `retrieve_similar_queries("user_00001", "Show me exciting action films", domain="movie", top_k=3)` correctly ranked “*Action movies*” (0.79 cosine) and “*Best action movies with high ratings*” (0.765) above “*Romantic comedies*” (0.371), excluding unrelated seeded queries entirely.

4 4b — Candidate Generation

`generate_candidates(query_text, similar_queries, behavioral_profile, domain, llm=None, num_candidates=5)` is the GQR/RA-GQR-style generation step.

4.1 Prompt construction

`build_prompt()` assembles one prompt from three inputs:

- the current query;
- the retrieved similar past queries from 4a, rendered as a bulleted list;
- a compact rendering of the Phase 3 behavioral divergence (`divergence["stated_top"] / ["actual_top"] / ["jaccard_similarity"]`), explicitly framed in the prompt text as “what this user says they want” versus “what this user actually engages with, which may differ.”

The LLM is asked for a **pool of 5** candidates, not the final 3 — deliberately more than needed, so that 4c’s filtering and 4d’s diversification have real material to work from rather than trivially keeping all 3 by construction. `_parse_suggestions()` then strips numbering, bullet characters, and quotation marks from the model’s line-per-suggestion response format.

4.2 LLM abstraction reuse

Rather than introducing a second provider-switching layer, this stage imports `nl2sparql.llm_interface.create_llm` directly from `web_app` (mirroring the same `sys.path` insertion pattern already used in `gui_v2.py`), per the roadmap’s own recommendation that recommendation generation should use “the same per-stage-configurable LLM abstraction already in `config.yaml`.” `generation.py`’s `_default_llm()` reads the `RECOMMENDER_LLM_PROVIDER` environment variable (default `gemini`) and an optional `RECOMMENDER_LLM_MODEL`, then delegates to `create_llm()` — so the recommender can run against Gemini, GPT (`openai`), Azure, or a local Mistral model without any code change, only an environment variable and that provider’s credentials. `llm` remains an injectable parameter so unit-level tests do not require a real API call.

A real bug was caught during verification and fixed here: `generation.py` now calls `load_dotenv()` itself (pointed at the repository-root `.env`), rather than relying on `web_app`’s `Config` class having already loaded it as a side effect. Calling `handle_recommend()` directly, without first constructing a `web_app Config` object, previously left the configured API key unset even though `.env` contained it, because nothing had triggered `load_dotenv()` yet. Since `recommender/` must work standalone (e.g. under `uvicorn recommender.api.app:app` with no `web_app` code running at all), it cannot depend on that side effect.

Verification. `build_prompt()` was called directly and its rendered text inspected, confirming all three input sections (current query, retrieved history, behavioral divergence) render correctly for `user_00001`’s real Phase 3 profile. `generate_candidates(..., llm=MockLLM())` with a hand-written mock returning a numbered/bulleted list confirmed `_parse_suggestions()` correctly strips numbering, bullets, and quotes down to 3 clean strings. Construction of the real `GeminiLLM()` object was also confirmed to succeed against the placeholder key present at the time, with `.generate()` failing cleanly with an `API_KEY_INVALID` error rather than crashing — i.e. the integration was correctly wired ahead of a real key being available (Section 8).

5 4c — Schema-Grounding Validation

5.1 What the roadmap originally specified

The roadmap called for running each candidate query’s implied entities and classes through the NL2SPARQL pipeline’s own `ClassRelevanceEvaluator/PropertyEvaluator` (or a lighter standalone check) to confirm it is answerable against the currently loaded domain schema before it is ever shown to a user — directly mitigating the hallucination/faithfulness risk documented in the LLM+KG roadmap survey (Pan et al.) and empirically observed in the supervisor’s own ADBIS 2026 paper (roughly 30% SPARQL-incorrectness on its factual question set).

5.2 Why the first two implementations were rejected

Attempt 1 — reuse `ClassRelevanceEvaluator`’s embedding mode directly. This method uses relative thresholding: it keeps every class scoring at or above mean – std across the schema’s own classes. With only 7–11 classes per domain, this reliably selects at least one class for *any* input, including completely unrelated text. Verified directly: evaluating “*Best pizza recipe*” against the Tourism schema returned a non-empty class list — i.e. “grounded: True” for a query with nothing to do with tourism.

Attempt 2 — an absolute cosine-similarity floor. Raw cosine similarity was computed between each candidate query and every class’s embedded description (same `all-mpnet-base-v2` model as 4a), requiring the maximum score across all classes to clear a fixed floor. This was calibrated against a small set of control (clearly off-topic) and probe (genuinely in-domain) phrases:

Query	Domain	Max cosine sim.	Best-matching class
Top rated action movies from the 2010s	movie	0.275	Movie
Movies directed by Christopher Nolan	movie	0.302	Movie
Documentaries produced in Canada	movie	0.288	Country
Best pizza recipe (<i>control</i>)	movie	0.175	SubscriptionPlan
How to fix a flat tire (<i>control</i>)	movie	0.094	Country
Churches in Verona	tourism	0.163	Location
Roman Churches in Verona	tourism	0.154	Location
Free Churches in Verona	tourism	0.092	Location
Events happening this weekend	tourism	0.401	Event
Art categories available	tourism	0.601	ArtCategory
Best pizza recipe (<i>control</i>)	tourism	0.064	Tour
How to fix a flat tire (control)	tourism	0.112	Tour
Latest news about the stock market (<i>control</i>)	tourism	0.004	—

Table 1: Absolute cosine-floor calibration data. The Movie domain separates cleanly, but the Tourism domain does not: the professor’s own canonical example, “*Free Churches in Verona*,” scores below an off-topic control phrase.

The Movie domain separates cleanly (in-domain floor ≈ 0.275 , control ceiling ≈ 0.175 — any threshold in between works). **The Tourism domain does not:** “*Free Churches in Verona*” — one of the supervisor’s own three canonical example suggestions, quoted verbatim in the professor’s original scoping email — scores 0.092, below the off-topic control phrase “*How to fix a flat tire*” at 0.112. No single threshold can accept the former while rejecting the latter. A z -score variant (relative to the query’s own mean/std across classes) was also tried and rejected

for the same reason: with only 7–11 classes, the maximum of such a small sample is reliably 1.3–2.3 standard deviations above the mean regardless of whether the query is on-topic, confirmed empirically — “*Best pizza recipe*” against the Movie schema scored $z = 2.31$, a *higher* z -score than several genuinely in-domain movie queries.

Root cause. Both domains’ machine-readable schema (m-schema) class descriptions are extremely sparse — generated purely from SHACL cardinality constraints (e.g. “*Art has exactly 1 Art Class ID. Art has at least 1 Art Name.*”), with no `rdfs:comment` descriptions and no `sh:example` instance values in either source `.ttl` file. The Tourism ontology’s `Art`/`ArtCategory` classes carry essentially no text a general-purpose sentence embedding could relate to English words like “church” — that correspondence only exists via the literal Italian category value “*Chiese*” in the live database, which is not present anywhere in the schema metadata being embedded. This is consistent with why the real NL2SPARQL pipeline’s own entity-linking stage exists as a separate, LLM-driven step: class-level embedding similarity alone was never designed to resolve this kind of gap on its own.

5.3 What shipped instead

Given this finding, `is_schema_grounded()` uses the absolute-cosine-floor mechanism from Attempt 2, but with `DEFAULT_THRESHOLD = 0.05` — low enough to reject only genuinely degenerate candidates (near-zero or negative similarity to *every* class in the schema, such as “*Latest news about the stock market*” at 0.004) while admitting everything else, including borderline-but-valid phrasings like “*Free Churches in Verona.*” `filter_grounded(candidates, domain)` applies it to a list, preserving order. This is explicitly documented, both in the module’s own docstring and here, as a coarse sanity filter, not the precision hallucination guard originally planned.

This is a genuine, reportable limitation for the thesis’s evaluation chapter: automatic schema-grounding of natural-language candidate suggestions, at the class-description level, was not achievable with acceptable precision given how sparse this project’s source ontologies are. A stronger version would require enriching class descriptions with real catalog/instance values pulled from the live Ontop SPARQL endpoints — a concrete, scoped follow-up, not attempted here.

Verification. The final `is_schema_grounded()` implementation was re-verified against 4 cases: “*Free Churches in Verona*” (tourism) $\rightarrow$ True, “*Roman Churches in Verona*” (tourism) $\rightarrow$ True, “*Top rated action movies*” (movie) $\rightarrow$ True, “*Latest news about the stock market*” (tourism) $\rightarrow$ False — all matching the expected outcome.

6 4d — Ranking and Diversity

`rank_and_diversify(candidates, similar_queries, behavioral_profile, top_n=3, diversity_penalty=0.5)` is the concrete mechanism through which the behavioral signal shapes the final output, echoing QRMOCGA’s (Barman et al.) two-objective framing without reimplementing its cooperative co-evolutionary genetic algorithm — explicitly out of scope per the roadmap’s own framing, with only its feature ideas borrowed.

6.1 Scoring

`score_candidate()` computes a weighted sum of two axes, each normalized to $[0, 1]$:

- `_history_similarity()` — the maximum Jaccard word overlap between a candidate and any of the user’s retrieved similar past queries (4a’s output);
- `_behavioral_alignment()` — the fraction of the behavioral profile’s **actual_top** interest categories (Phase 3) whose words appear in the candidate text. This is the concrete place the thesis’s stated-vs-actual signal enters ranking: a candidate that textually references what the user *actually watches*, not just what they typed, scores higher on this axis.

6.2 Diversification

A greedy, Maximal-Marginal-Relevance-style loop repeatedly selects the highest-scoring remaining candidate, penalized by its maximum Jaccard word overlap with candidates already selected (`diversity_penalty=0.5`), so the final three suggestions are not near-duplicates of one another.

Verification. A hand-built pool of 5 candidates (one near-duplicate of query history, two behaviorally-aligned, one off-axis, one more history-similar) was fed through `rank_and_diversify(..., top_n=3)`. It correctly picked the history-similar candidate first, then diversified into the two behaviorally-aligned candidates rather than a near-duplicate of the first pick.

7 End-to-End Verification

7.1 Full chain, mocked LLM

`recommender.core.generation._default_llm` was patched with a mock returning 5 fixed candidates, and the real `recommender.api.service.handle_recommend()` — the actual production code path, not a reimplementation — was called for `user_00001 / movie / “Show me exciting action films.”` Result:

Listing 1: Mocked-LLM end-to-end output, ranked top to bottom.

War documentaries produced in the USA
Crime thrillers with high ratings
Adventure movies set during wartime

The top-ranked suggestion is the one most strongly aligned with `user_00001`’s *actual* top interest categories (Movie, USA, Canada, Adventure, War, per their Phase 3 profile) rather than a plain paraphrase of the stated query about “action films” — a concrete, working demonstration of the thesis’s central premise.

7.2 Full chain, real HTTP, placeholder key

`uvicorn recommender.api.app:app` was started and the same request sent over real HTTP. The server log confirmed the full path executed (log → retrieval → generation attempt → the

same `API_KEY_INVALID` failure as above, caught and logged as a warning, not a crash), and the HTTP response was 200 OK with `{"suggestions": []}` — i.e. the degrade-gracefully behavior inside `handle_recommend()`'s exception handling around candidate generation works over the real transport, not only in-process.

8 Real-LLM Verification: Gemini/OpenAI Blocked, Ollama Used Instead

Both configured cloud providers turned out to be dead ends in this environment, for account reasons unrelated to the code: the `OPENAI_API_KEY` in `.env` returns `insufficient_quota` (no billing configured on the account), and the `GEMINI_API_KEY` returns `RESOURCE_EXHAUSTED` with a zero-quota limit for every model’s free tier (the key’s Google project has no free-tier allowance enabled).

Rather than block on that, a new `ollama` provider was added to `web_app/nl2sparql/llm_interface.py` (an `OllamaLLM` class using `requests` against Ollama’s `/api/generate` HTTP endpoint — no API key, no quota) and wired into `create_llm()` alongside the existing `gemini/mistral/openai/azure` providers. This was chosen over the already-existing local `MistralLLM` class because that class depends on `llama-cpp-python` (a non-trivial C++ build on Windows) plus a manually-downloaded, low-quality quantized GGUF file; Ollama is a single Docker image with pre-optimized quantization and no compilation step. A new `ollama` service was added to `docker-compose.yml`, and `llama3.1:8b` (4.9 GB) was pulled into the running container.

8.1 Verified against real output

Both `generate_candidates()` directly, and the full production path `handle_recommend()` (the same function the FastAPI endpoint and the development harness both call), were run for `user_00001` / movie domain / “*Show me exciting action films.*” The `handle_recommend()` result:

Listing 2: End-to-end suggestions from genuine `llama3.1:8b` output, via the real production code path.

```
{
  "suggestions": [
    "War movies with high ratings on IMDB",
    "Movies that combine action and adventure genres",
    "Films starring actors from Canada or USA"
  ]
}
```

The top-ranked suggestion references *War*, and the other two reference *Canada/USA* — all three are in `user_00001`’s behavioral-profile `actual_top` categories (`Movie`, `USA`, `Canada`, `Adventure`, `War`), not a paraphrase of the stated query about “action films.” This is the same qualitative pattern already observed with the mocked-LLM run above, now reproduced with genuine model output.

Latency. Approximately 18–22 seconds per `/recommend` call on this machine (CPU-only Docker container, no GPU passthrough) — acceptable for offline evaluation runs, but a real deployment-latency concern distinct from suggestion quality, not addressed here.

Remaining limitation. This validates the *pipeline*, not the *provider* originally planned (Gemini). If a funded Gemini or OpenAI key becomes available, `RECOMMENDER_LLM_PROVIDER` can be flipped back with no code changes — the abstraction was already provider-agnostic before this change; Ollama is an additional option, not a replacement.

9 Interpretation and Limitations

- **The 4c grounding check is intentionally weak** (Section 5). This should be reported in the thesis as an explicit, empirically-justified limitation, not silently shipped as if it were

the precision filter originally planned. The calibration data in Table 1 is itself genuine results-chapter material: it demonstrates a real failure mode (sparse ontology metadata undermining embedding-based grounding) with a concrete example query supplied directly by the supervisor.

- **4b is now verified against real LLM output** (Section 8), via a local Ollama model rather than the originally-planned Gemini — both cloud keys available in this environment hit billing/quota walls unrelated to the code. Suggestion quality from `llama3.1:8b` looks sound on the worked example shown; a systematic quality check across more users and queries is Phase 5’s job, not this phase’s.
- **`num_candidates=5` (the pool size 4b requests) is an arbitrary choice**, not derived from data — worth a sensitivity check (e.g. pool sizes of 4/6/8) once real LLM output is routinely available, since a larger pool gives 4d’s diversification more to work with at the cost of more tokens per request.
- **The behavioral-alignment axis in 4d only works for the Movie domain today**, since Phase 3 profiles only exist for Movie users (per the Phase 0 decision that Tourism stays catalog-only). `_behavioral_alignment()` correctly returns 0.0 when no profile exists, so Tourism recommendations degrade to history-similarity-only ranking rather than erroring — but this means the thesis’s core stated-vs-actual contribution is only demonstrable end-to-end on the Movie domain, consistent with every earlier phase’s own scoping note.

10 Summary and Next Steps

Phase 4 delivers a verified, four-stage query recommender — retrieval, LLM generation, schema-grounding validation, and behavior-aware ranking — composed entirely inside the extension point Phase 2 left open, `handle_recommend()`, with no changes to the API contract or the development harness. It was verified structurally (mocked LLM, real HTTP with a placeholder key) and, after both configured cloud providers turned out to be blocked by account/billing issues, with genuine output from a locally hosted Ollama model, producing suggestions that concretely track a user’s real watching behavior rather than merely paraphrasing their typed query.

Phase 5 (documented separately in the *Phase 5 Implementation Report*) is the direct consumer of this phase’s output: a controlled ablation isolating the effect of the behavioral signal on ranking (ranking with vs. without `behavioral_profile`), a proxy-recall check against held-out watch history, and a non-LLM classical baseline for comparison. Full detail on Phase 5 is tracked in the project roadmap document, *Thesis Project Plan*.